

Angela Wei

Data 260

Homework 5

Section 0:

Hardware Used: HP Pavilion Windows 11 Pro

Local Models Used: Local Host in port base 8439, FastAPI, FastMCP

Commit Hash: 96bdef93aa7befc6920cd250c314a0fb7c5b79d1

Personal Configuration and Domain

SID4: 1339

PORT_BASE: 8439

PREFIX: s1339

SEED: 1339

VERIFY_SEED: 261339

DOMAIN_ID: 3 – Grocery Supply and Recall Notices

Part 1

Database set up and password hash configuration in the backend

```
DATABASE_URL = "mysql+pymysql://username:password@localhost:8439/my_db_info"

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
```

Creating a session model based on the user's

```
class SessionModel(Base):
    __tablename__ = "session_info"

    session_id = Column(String(255), primary_key=True)
    user_id = Column(Integer, ForeignKey("user_info.username_id", ondelete="CASCADE"), nullable=False)
    time_created = Column(DateTime, default=datetime.datetime.utcnow())
    time_expiration = Column(DateTime, nullable=False)
```

Setting up HTTP status codes based on the user authentication.

```
def get_current_user(request: Request, db: Session = Depends(get_db)):
    session_token = request.cookies.get("session_token")
    if not session_token:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Welcome! Please login"
        )

    db_session = db.query(SessionModel).filter(SessionModel.session_id == session_token).first()

    if not db_session or db_session.time_expiration < datetime.datetime.utcnow():
        if db_session:
            db.delete(db_session)
            db.commit()
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Your session has expired, please login again"
        )

    return db_session.user_id
```

```
def login(credentials: LoginSchema, response: Response, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.email == credentials.email).first()

    if not user or not pwd_context.verify(credentials.password, user.user_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid email or password"
        )

    token = secrets.token_hex(32)
    expires = datetime.datetime.utcnow() + datetime.timedelta(minutes=30)

    # Updated to match DB column names: time_expiration
    new_session = SessionModel(
        session_id=token,
        user_id=user.username_id,
        time_expiration=expires
    )
    db.add(new_session)
    db.commit()
```

Deleting a user record from the database.

```
@router.delete("/records/{record_id}")
def delete_record(record_id: int, user_id: int = Depends(get_current_user), db: Session = Depends(get_db)):
    record = db.query(Record).filter(Record.id == record_id).first()
    if not record:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Error: Cannot find record")

    db.delete(record)
    db.commit()
    return {"message": "Successfully deleted user record"}
```

One of several unsuccessful to run everything in the Port Base.

```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4->code> python hw3_main.py
Traceback (most recent call last):
  File "C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4->code\hw3_main.py", line 7, in <module>
    from hw3_authentication import router as auth_router
  File "C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4->code\hw3_authentication.py", line 75, in <module>
    Base.metadata.create_all(bind=engine)
  File "C:\Users\wei_1\AppData\Roaming\Python\Python312\site-packages\sqlalchemy\sql\schem.py", line 5929, in create_all
    bind._run_ddl_visitor(
  File "C:\Users\wei_1\AppData\Roaming\Python\Python312\site-packages\sqlalchemy\engine\base.py", line 3269, in _run_ddl_visitor
    with self.begin() as conn:
  File "C:\Python312\Lib\contextlib.py", line 137, in __enter__
    return next(self.gen)
           ^^^^^^^^^^^^^^^
  File "C:\Users\wei_1\AppData\Roaming\Python\Python312\site-packages\sqlalchemy\engine\base.py", line 3259, in begin
    with self.connect() as conn:
           ^^^^^^^^^^^^^^^
  File "C:\Users\wei_1\AppData\Roaming\Python\Python312\site-packages\sqlalchemy\engine\base.py", line 3295, in connect
    return self._connection_cls(self)
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\wei_1\AppData\Roaming\Python\Python312\site-packages\sqlalchemy\engine\base.py", line 144, in __init__
    self._dbapi_connection = engine.raw_connection()
                             ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\wei_1\AppData\Roaming\Python\Python312\site-packages\sqlalchemy\engine\base.py", line 3319, in raw_connection
    return self.pool.connect()
           ^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\wei_1\AppData\Roaming\Python\Python312\site-packages\sqlalchemy\pool\base.py", line 448, in connect
    return _ConnectionRecord._checkout(self)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\wei_1\AppData\Roaming\Python\Python312\site-packages\sqlalchemy\pool\base.py", line 1272, in _checkout
    fairy = _ConnectionRecord.checkout(pool)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Part 2

Part 2A

Creating the mcp meals folder and setting up the virtual environment

```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> mkdir mcp_meals

Directory: C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code

Mode                LastWriteTime         Length Name
----                -
d-----         10/4/2026   5:16 PM             mcp_meals

PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> cd mcp_meals
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\mcp_meals> python -m venv hw5_venv
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\mcp_meals>

PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\mcp_meals> hw5_venv\Scripts\Activate.ps1
(hw5_venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\mcp_meals>
```

Everything related to MCP and the tools and testing will be done in this folder

Start backup > ... Homework > data260--SID4- > code > mcp_meals >

			 Sort ▾	 View ▾	...
☐ Name	Date modified	Type	Size		
 hw5_venv	10/4/2026 5:16 PM	File folder			
 mcp_meals_server.py	10/4/2026 5:30 PM	Python Source File	3 KB		

Installing MCP related packages so all MCP related files will run properly

```
(hw5_venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\mcp_meals> pip install mcp ht
tpx
Collecting mcp
  Obtaining dependency information for mcp from https://files.pythonhosted.org/packages/20/d7/4a2878f128df729db3ca04e71bfce26405c
c2e0c9ebe19c592feeb4c61b9/mcp-2.3.0-py3-none-any.whl.metadata
  Downloading mcp-2.3.0-py3-none-any.whl.metadata (6.6 kB)
Collecting httpx
  Obtaining dependency information for httpx from https://files.pythonhosted.org/packages/2a/39/e50c7c3a983047577ee07d2a9e53faf5a
69493943ec3f6a384bdc792deb2/httpx-0.28.1-py3-none-any.whl.metadata
  Downloading httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting anyio>=4.9 (from mcp)
  Obtaining dependency information for anyio>=4.9 from https://files.pythonhosted.org/packages/12/b8/4bd346e22b28902df4d651910f52
42c28d84e4a5c2435ca5c3f797ed7e2e/anyio-4.15.1-py3-none-any.whl.metadata
  Downloading anyio-4.15.1-py3-none-any.whl.metadata (4.7 kB)
Collecting httpx2>=2.10.0 (from mcp)
  Obtaining dependency information for httpx2>=2.10.0 from https://files.pythonhosted.org/packages/d8/9c/6fe8931fd9f381042a9e4c7d
5a7b4cbf7016b252bec0c99a49fce42c3326/httpx2-2.13.1-py3-none-any.whl.metadata
  Downloading httpx2-2.13.1-py3-none-any.whl.metadata (9.8 kB)
Collecting jsonschema>=4.20.0 (from mcp)
  Obtaining dependency information for jsonschema>=4.20.0 from https://files.pythonhosted.org/packages/69/90/f63fb5873511e014207a
475e2bb4e8b2e570d655b00ac19a9a0ca0a385ee/jsonschema-4.26.0-py3-none-any.whl.metadata
  Using cached jsonschema-4.26.0-py3-none-any.whl.metadata (7.6 kB)
Collecting mcp-types==2.3.0 (from mcp)
  Obtaining dependency information for mcp-types==2.3.0 from https://files.pythonhosted.org/packages/ce/48/32583066b155bb5d4507e7
887dc88ee23118cc4de8e2a12870c49719e2a8/mcp-types-2.3.0-py3-none-any.whl.metadata
  Downloading mcp-types-2.3.0-py3-none-any.whl.metadata (1.9 kB)
Collecting opentelemetry-api>=1.28.0 (from mcp)
  Obtaining dependency information for opentelemetry-api>=1.28.0 from https://files.pythonhosted.org/packages/44/b9/040d1a1c78369
```

```
(hw5_venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\mcp_meals> pip install "mcp[c
li]"
Requirement already satisfied: mcp[cli] in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\data260--sid4-\code\
mcp_meals\hw5_venv\lib\site-packages (2.3.0)
Requirement already satisfied: anyio>=4.9 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\data260--sid4-\cod
e\mcp_meals\hw5_venv\lib\site-packages (from mcp[cli]) (4.15.1)
Requirement already satisfied: httpx2>=2.10.0 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\data260--sid4-
\code\mcp_meals\hw5_venv\lib\site-packages (from mcp[cli]) (2.13.1)
Requirement already satisfied: jsonschema>=4.20.0 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\data260--s
id4-\code\mcp_meals\hw5_venv\lib\site-packages (from mcp[cli]) (4.26.0)
Requirement already satisfied: mcp-types==2.3.0 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\data260--sid
4-\code\mcp_meals\hw5_venv\lib\site-packages (from mcp[cli]) (2.3.0)
Requirement already satisfied: opentelemetry-api>=1.28.0 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\dat
a260--sid4-\code\mcp_meals\hw5_venv\lib\site-packages (from mcp[cli]) (1.45.0)
Requirement already satisfied: pydantic>=2.12.0 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\data260--sid
4-\code\mcp_meals\hw5_venv\lib\site-packages (from mcp[cli]) (2.13.5)
Requirement already satisfied: pyjwt[crypto]>=2.10.1 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\data260
--sid4-\code\mcp_meals\hw5_venv\lib\site-packages (from mcp[cli]) (2.15.1)
Requirement already satisfied: python-multipart>=0.0.9 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\data2
60--sid4-\code\mcp_meals\hw5_venv\lib\site-packages (from mcp[cli]) (0.0.32)
Requirement already satisfied: pywin32>=311 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\data260--sid4-\c
ode\mcp_meals\hw5_venv\lib\site-packages (from mcp[cli]) (312)
Requirement already satisfied: sse-starlette>=3.0.0 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\data260-
--sid4-\code\mcp_meals\hw5_venv\lib\site-packages (from mcp[cli]) (3.5.0)
Requirement already satisfied: starlette>=0.27 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\data260--sid4
-\code\mcp_meals\hw5_venv\lib\site-packages (from mcp[cli]) (1.7.0)
Requirement already satisfied: typing-extensions>=4.13.0 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\dat
a260--sid4-\code\mcp_meals\hw5_venv\lib\site-packages (from mcp[cli]) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.1 in c:\users\wei_1\desktop\angelawei\angela-projects\data260\homework\data
```

Tools running successfully in the browser

```
(hw5_venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\mcp_meals> mcp-inspector pyth
on mcp_meals_server.py
Starting MCP inspector...

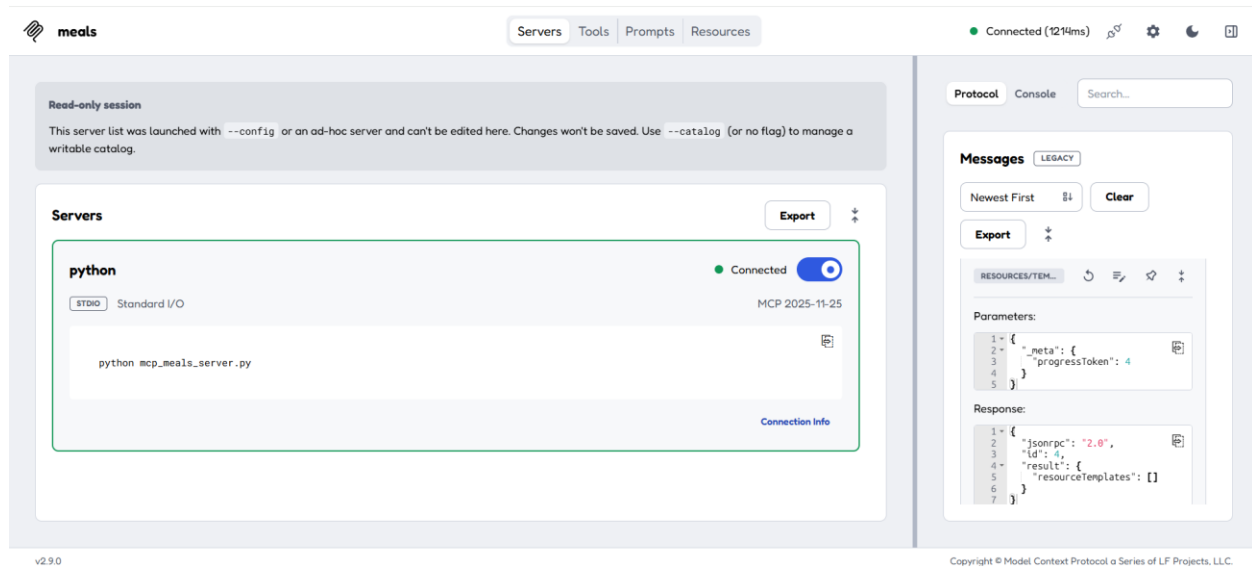
MCP Inspector Web is up and running at:
  http://127.0.0.1:6274?MCP_INSPECTOR_API_TOKEN=3f9c03bee8c4e4a13fd4d147fe3fd6e472595620a5c03c040e2958a662437f3f

  Sandbox (MCP Apps): http://127.0.0.1:6275/sandbox

  Auth token: 3f9c03bee8c4e4a13fd4d147fe3fd6e472595620a5c03c040e2958a662437f3f

  Secrets: OS keychain

Opening browser...
```



Tool in code to identify meal by name.

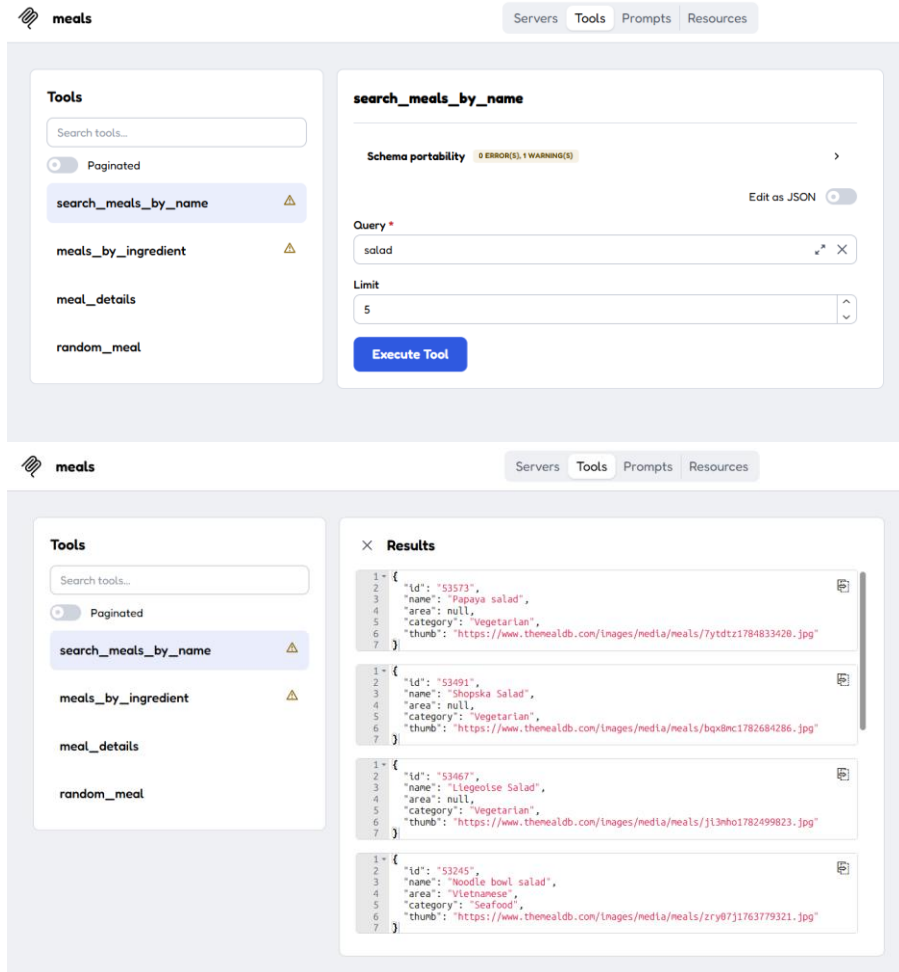
```
async def search_meals_by_name(query: str, limit: int = 5) -> list | str:
    async with httpx.AsyncClient() as client:
        response = await client.get(f"{get_db}/search.php", params={"s": query})
        data = response.json()

        if not data.get("meals"):
            return "No matches found."

        results = []

        for meal in data["meals"][:limit]:
            results.append({"id": meal.get("idMeal"), "name": meal.get("strMeal"), "area": meal.get("strArea"),
                           "category": meal.get("strCategory"), "thumb": meal.get("strMealThumb")})
```

Successfully searching a meal by name



Tool in code to search for ingredients used.

```
async def meals_by_ingredient(ingredient: str, limit: int = 12) -> list | str:
    async with httpx.AsyncClient() as client:
        response = await client.get(f"{get_db}/filter.php", params={"i": ingredient})
        data = response.json()

        if not data.get("meals"):
            return "No matches found."

        results = []

        for meal in data["meals"][0:limit]:
            results.append({"id": meal.get("idMeal"), "name": meal.get("strMeal"), "thumb": meal.get("strMealThumb")})

        return results
```

Unsuccessful ingredient search.

The screenshot shows the 'meals' application interface. On the left, a 'Tools' sidebar lists search tools: 'search_meals_by_name', 'meals_by_ingredient' (selected), 'meal_details', and 'random_meal'. The main panel is titled 'meals_by_ingredient'. It shows 'Schema portability' with 0 errors and 1 warning. The 'Ingredient' field contains 'curry'. The 'Limit' is set to 12. An 'Execute Tool' button is at the bottom. On the right, the 'Server Console' shows a log of a request to 'https://www.themealdb.com/api/json/v1/1/search.php?s=salad'.

meals

Servers Tools Prompts Resources

Connected (124ms)

Tools

Search tools...

Paginated

search_meals_by_name

meals_by_ingredient

meal_details

random_meal

meals_by_ingredient

Schema portability 0 ERRORS, 1 WARNING

Edit as JSON

Ingredient *

curry

Limit

12

Execute Tool

Protocol Console Search...

Server Console

Newest First Clear

Export

request of type
server.py:733

CallToolRequest

6:28:48 PM [18/04/26 18:28:48]

6:28:09 PM INFO HTTP Request:
GET
_client.py:1748

https://www.themealdb.co
m/api/json

/v1/1/search.php?s=salad

HTTP/1.1

v2.9.0

Copyright © Model Context Protocol a Series of LF Projects, LLC

Successful ingredient search

The screenshot shows the 'meals' application interface. On the left, the 'Tools' sidebar is the same. The main panel is titled 'Results'. It shows 'No matches found.' and a 'Structured Output' section with a JSON response: {'result': 'No matches found.'}. On the right, the 'Server Console' shows a log of a request to 'https://www.themealdb.com/api/json/v1/1/search.php?s=salad'.

meals

Servers Tools Prompts Resources

Connected (124ms)

Tools

Search tools...

Paginated

search_meals_by_name

meals_by_ingredient

meal_details

random_meal

Results

No matches found.

Structured Output

```
{
  "result": "No matches found."
}
```

Protocol Console Search...

Server Console

Newest First Clear

Export

request of type
server.py:733

CallToolRequest

6:28:48 PM [18/04/26 18:28:48]

6:28:09 PM INFO HTTP Request:
GET
_client.py:1748

https://www.themealdb.co
m/api/json

/v1/1/search.php?s=salad

HTTP/1.1

v2.9.0

Copyright © Model Context Protocol a Series of LF Projects, LLC

Tools

Search tools...

☒ Paginated

search_meals_by_name

meals_by_ingredient

meal_details

random_meal

meals_by_ingredient

Schema portability 0 ERROR(S), 1 WARNING(S)

Edit as JSON

Ingredient *

flour

Limit

12

Execute Tool

Tools

Search tools...

☒ Paginated

search_meals_by_name

meals_by_ingredient

meal_details

random_meal

Results

```
1 {
2   "id": "53120",
3   "name": "Ablesklver",
4   "thumb": "https://www.themealdb.com/images/media/meals/wkhg581762773124.jpg"
5 }
```

```
1 {
2   "id": "53049",
3   "name": "Apan ballk",
4   "thumb": "https://www.themealdb.com/images/media/meals/adxcbq1619787919.jpg"
5 }
```

```
1 {
2   "id": "53575",
3   "name": "Apple Pie",
4   "thumb": "https://www.themealdb.com/images/media/meals/stnxzp1784835840.jpg"
5 }
```

```
1 {
2   "id": "53396",
3   "name": "Baghlaw-e-Khanagi",
4   "thumb": "https://www.themealdb.com/images/media/meals/bjtjyl1779552068.jpg"
5 }
```

```
1 {
2   "id": "53456",
3   "name": "Bajan Salt Bread Recipe",
4   "thumb": "https://www.themealdb.com/images/media/meals/sr7chd1780264758.jpg"
5 }
```

```
1 {
```


Tool in code to search for meal ID.

```
async def meal_details(id: str) -> dict | str:
    async with httpx.AsyncClient() as client:
        response = await client.get(f"{get_db}/lookup.php", params={"i": id})
        data = response.json()

        if not data.get("meals"):
            return "No matches found."

        return format_full_meal(data["meals"][0])
```

Searching a meal by its ID.

The image shows two screenshots of a web application interface for 'meals'. The top screenshot shows the 'Tools' section on the left with a list of tools: 'search_meals_by_name', 'meals_by_ingredient', 'meal_details' (selected), and 'random_meal'. The 'meal_details' tool is highlighted. The right panel shows the 'meal_details' tool configuration with a text input field labeled 'Id *' containing the value '53573'. Below the input field is a blue button labeled 'Execute Tool'. The bottom screenshot shows the same interface after the tool has been executed. The right panel now displays the 'Results' section, which contains a JSON object representing the meal details for ID '53573'. The JSON object includes fields for 'id', 'name', 'category', 'area', 'instructions', 'image', 'source', 'youtube', and 'ingredients'. The 'ingredients' field is an array of objects, each containing 'name' and 'measure'.

meal_details

Id *

53573

Execute Tool

Results

```
{
  "id": "53573",
  "name": "Papaya salad",
  "category": "Vegetarian",
  "area": null,
  "instructions": "Cut, peel and shred papaya. I used a hand julienne shredder. If y",
  "image": "https://www.thenealdb.com/images/media/meals/7ytdtz1784833428.jpg",
  "source": "https://doninacagourmet.com/green-papaya-salad-with-fried-snapper/",
  "youtube": "",
  "ingredients": [
    {
      "name": "Green Papaya",
      "measure": "1/2"
    },
    {
      "name": "Lemon Juice",
      "measure": "1 tablespoon"
    },
    {
      "name": "Salt",
      "measure": "To taste"
    },
    {
      "name": "Carrots",
      "measure": "1"
    },
    {
      "name": "Cherry Tomatoes",
      "measure": "1 cup"
    },
    {
      "name": "Spring Onions",
      "measure": "1"
    }
  ]
}
```

Tool in code to search for meal ID.

```
async def random_meal() -> dict | str:
    async with httpx.AsyncClient() as client:
        response = await client.get(f"{get_db}/random.php")
        data = response.json()

        if not data.get("meals"):
            return "No matches found."

        return format_full_meal(data["meals"][0])
```

Randomly generating meal information.

The image shows two screenshots of a web application interface for 'meals'. The top screenshot shows the 'Tools' panel on the left with 'random_meal' selected. The 'random_meal' tool panel on the right has an 'Execute Tool' button. The bottom screenshot shows the same interface after execution, with the 'Results' panel displaying a JSON object for a meal.

Tools Panel (Left):

- Search tools...
- ☐ Paginated
- search_meals_by_name
- meals_by_ingredient
- meal_details
- random_meal**

random_meal Panel (Right):

random_meal

Edit as JSON ☐

Execute Tool

Results Panel (Bottom):

```
1 {
2   "id": "53158",
3   "name": "Air fryer patatas bravas",
4   "category": "Vegetarian",
5   "area": "Spanish",
6   "instructions": "Step 1\r\nSoak the potatoes in just-boiled water for 30 mins, the",
7   "image": "https://www.themeatdb.com/images/media/meals/3n8yae1763257951.jpg",
8   "source": "https://www.bbcgoodfood.com/recipes/air-fryer-patatas-bravas",
9   "youtube": "https://www.youtube.com/watch?v=kzLaw6PG2UY",
10  "ingredients": [
11    {
12      "name": "Potatoes",
13      "measure": "980g"
14    },
15    {
16      "name": "Olive Oil",
17      "measure": "3 tablespoons"
18    },
19    {
20      "name": "Onion",
21      "measure": "1 chopped"
22    },
23    {
24      "name": "Garlic",
25      "measure": "1 clove peeled crushed"
26    },
27    {
28      "name": "Paprika",
29      "measure": "1 tblsp"
30    },
31    {
32      "name": "Tomato Puree",
```

Part 2B

Successfully starting up tools related to own domain: grocery recall

```
(hw5.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\mcp_meals> mcp-inspector python mcp_GroceryRecall_rel.py
Starting MCP inspector...

MCP Inspector Web is up and running at:
http://127.0.0.1:6274?MCP_INSPECTOR_API_TOKEN=9971babce1911e142e7143ea0edfb49b51d4b36152d5cebd9e94a8d0ac4c6c0

Sandbox (MCP Apps): http://127.0.0.1:6275/sandbox

Auth token: 9971babce1911e142e7143ea0edfb49b51d4b36152d5cebd9e94a8d0ac4c6c0

Secrets: OS keychain

Opening browser...
```

Tool in code to identify contamination recalls.

```
async def search_grocery_recalls(query: str) -> dict:
    try:
        if not query or not query.strip():
            return create_envelope(error = "Please enter a product or brand name to search recalls!")

        async with httpx.AsyncClient() as client:
            response = await client.get(f"{get_db}?search=product_description:{query}&limit=5")

            if response.status_code == 404:
                return create_envelope(error = "No matching grocery recalls found! Try again.")

            res_json = response.json()
            results = res_json.get("results", [])

            if not results:
                return create_envelope(error = "No matching grocery recalls found! Try again.")

            recall_matches = [
                {
                    "recall_number": r.get("recall_number"),
                    "product_description": r.get("product_description"),
                    "reason_for_recall": r.get("reason_for_recall"),
                    "company_name": r.get("recalling_firm"),
                    "recall_date": r.get("report_date")
                } for r in results
            ]

            return create_envelope(data = recall_matches)

    except Exception as e:
        return create_envelope(error = f"Error fetching data: {str(e)}")
```

Successfully identifying contamination recall

The top screenshot shows the 'GroceryRecall_rel' application interface. On the left, a 'Tools' sidebar lists 'search_grocery_recalls', 'recall_detail_lookup', and 'aggregate_company_stats'. The main panel is titled 'search_grocery_recalls' and features a search bar with the query 'contamination' and an 'Execute Tool' button. An 'Edit as JSON' toggle is also present.

The bottom screenshot shows the 'Results' panel of the same application. It displays a JSON response with two recall entries:

```
1 {
2   "ok": true,
3   "data": [
4     {
5       "recall_number": "P-0975-2021",
6       "product_description": "Purina Health Products: EpiGenius Klds Creamy Chocolate",
7       "reason_for_recall": "Product recalled due to undeclared milk in MCT powder us",
8       "company_name": "Organic By Nature, Inc.",
9       "recall_date": "20210922"
10    },
11    {
12       "recall_number": "P-1772-2018",
13       "product_description": "Sunny Side Farms Peanut Butter Cup Ice Cream: 48 oz.",
14       "reason_for_recall": "Some containers of ice cream in a Peanut Butter Cup flav",
15       "company_name": "Super Store Industries-Turlock Dairy Division",
16       "recall_date": "20180822"
17    }
18  ],
19   "error": null
20 }
```

Tool in code to identify recall ID

```
async def recall_detail_lookup(recall_number: str) -> dict:
    try:
        if not recall_number or not recall_number.strip():
            return create_envelope(error = "Please enter a recall number to continue!")

        async with httpx.AsyncClient() as client:
            #search by a recall number
            response = await client.get(f"{get_db}?search=recall_number:{recall_number}")

            if response.status_code == 404:
                return create_envelope(error = "Could not find any information on this recall number.")

            res_json = response.json()
            results = res_json.get("results", [])

            if not results:
                return create_envelope(error = "Could not find any information on this recall number.")

            data = results[0]
            recall_details = {
                "recall_number": data.get("recall_number"),
                "product_description": data.get("product_description"),
                "reason_for_recall": data.get("reason_for_recall"),
                "company_name": data.get("recalling_firm"),
                "distribution_pattern": data.get("distribution_pattern"),
                "recall_date": data.get("report_date"),
                "status": data.get("status")
            }

            return create_envelope(data = recall_details)
```

Successfully identifying a recall by its ID number

The top screenshot shows the 'GroceryRecall_rel' application interface. On the left, the 'Tools' sidebar lists 'search_grocery_recalls', 'recall_detail_lookup' (selected), and 'aggregate_company_stats'. The main panel, titled 'recall_detail_lookup', has a search bar with 'F-0975-2021' and an 'Execute Tool' button. An 'Edit as JSON' toggle is also present.

The bottom screenshot shows the 'Results' panel after execution. It displays a JSON object with the following structure:

```
1 {
2   "ok": true,
3   "data": {
4     "recall_number": "F-0276-2017",
5     "product_description": "CytoDetox, Hydrolyzed Clinoptilolite Fragments, 1 oz./30
6     "reason_for_recall": "Recall initiated as a precautionary measure due to potenti
7     "company_name": "Pharmatech LLC",
8     "distribution_pattern": "FL, MI, MS, and OH.",
9     "recall_date": "20161102",
10    "status": "Terminated"
11  },
12  "error": null
13 }
```

Tool in code to identify recalls by company

```
async def aggregate_company_stats(company_name: str) -> dict:
    try:
        if not company_name or not company_name.strip():
            return create_envelope(error = "Please enter a company name to continue!")

        async with httpx.AsyncClient() as client:
            #search OpenFDA by recalling firm name
            response = await client.get(f"{get_db}?search=recalling_firm:{company_name}&limit=100")

            if response.status_code == 404:
                return create_envelope(error = "Could not find any recall data for this company.")

            res_json = response.json()
            company_info = res_json.get("results", [])

            if not company_info:
                return create_envelope(error = "Could not find any recall data for this company.")

            total_recalls = len(company_info)
            product_list = [r.get("product_description")[:50] + "..." for r in company_info if r.get("product_description")]

            company_summary = {
                "company_name": company_name,
                "total_recalls_recorded": total_recalls,
                "recent_products_recalled": product_list[:5]
            }

            return create_envelope(data = company_summary)

    except Exception as e:
        return create_envelope(error = str(e))
```

Tools

Search tools...

Paginated

search_grocery_recalls

recall_detail_lookup

aggregate_company_stats

aggregate_company_stats

Edit as JSON

Company Name *

SafeWAY

↕

✕

Execute Tool

 GroceryRecall_rel

Servers

Tools

Prompts

Resources

Tools

Paginated

search_grocery_recalls

recall_detail_lookup

aggregate_company_stats

×

Results

```
1 {
2   "ok": true,
3   "data": {
4     "company_name": "Safeway",
5     "total_recalls_recorded": 16,
6     "recent_products_recalled": [
7       "Little Bear Fresh Kale, Net Wt. 16oz...",
8       "Angel Food Cake Fruit Topped; Labeled for retail...",
9       "Safeway Select French Salted Caramel (Fleur De Sel...",
10      "Sunbyside Farms, diced Organic Butternut Squash, ...",
11      "Robinson Fresh, Turnip Greens, 1lb. packages / B...",
12    ]
13  },
14  "error": null
15 }
```

Purposeful failed recall (entering a movie name instead of grocery or recall related term).

 GroceryRecall_rel

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_grocery_recalls

recall_detail_lookup

aggregate_company_stats

search_grocery_recalls

Query *

The Avengers

Execute Tool

Edit as JSON

Tools

Search tools...

Paginated

search_grocery_recalls

recall_detail_lookup

aggregate_company_stats

Results

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "No matching grocery recalls found! Try again."
5 }
```

Entering a random number instead of a proper ID number.

 GroceryRecall_rel

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_grocery_recalls

recall_detail_lookup

aggregate_company_stats

recall_detail_lookup

Edit as JSON

Recall Number *
67

Execute Tool

×

Results

```
1 {  
2   "ok": false,  
3   "data": null,  
4   "error": "Could not find any information on this recall number."  
5 }
```


Entering a non-grocery related brand/company name.

 GroceryRecall_rel

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_grocery_recalls

recall_detail_lookup

aggregate_company_stats

aggregate_company_stats

aggregate_company_stats

Company Name *

Pokemon

Execute Tool

Results

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "Could not find any recall data for this company."
5 }
```

Part 3

Used of verify_seed to test and simulate different failure rates. Folder and path of which final JSON output will be written to is identified.

```
verify_seed = 261339

rates = [0.0, 0.2, 0.5]

output_directory = Path(__file__).resolve().parents[2] / "reports" / "hw05" / "raw"
output_directory.mkdir(parents = True, exist_ok = True)

target_test = (search_grocery_recalls, {"query": "cheese"})

summary_results = []

for rate in rates:
    latencies = []
    successes = 0
    raw_logs = []

    for i in range(50):
        call_seed = verify_seed + int(rate * 100) + i
        func, args = target_test
        res = call_retry(func, args, fail_rate = rate, seed = call_seed)

        raw_logs.append(res)
        if res["success"]:
            successes += 1
            latencies.append(res["latency_ms"])

    success_rate = (successes / 50) * 100
    mean_latency = sum(latencies) / len(latencies) if latencies else 0
    latencies.sort()
    p99_latency = latencies[int(len(latencies) * 0.99)] if latencies else 0

    summary_results.append({"injected_failure_rate": f"{int(rate * 100)}%", "success_rate": f"{success_rate}%",
                           "mean_latency_ms": round(mean_latency, 2), "p99_latency_ms": round(p99_latency, 2)})

    with open(output_directory / f"grocery_failure_rate_{int(rate * 100)}.json", "w") as f:
        json.dump(raw_logs, f, indent=2)

print(json.dumps(summary_results, indent=2))
```

Final JSON dump after the tests are finished.

```
success_rate = (successes / 50) * 100
mean_latency = sum(latencies) / len(latencies) if latencies else 0
latencies.sort()
p99_latency = latencies[int(len(latencies) * 0.99)] if latencies else 0

summary_results.append({"injected_failure_rate": f"{int(rate * 100)}%", "success_rate": f"{success_rate}%",
                        "mean_latency_ms": round(mean_latency, 2), "p99_latency_ms": round(p99_latency, 2)})

with open(output_directory / f"grocery_failure_rate_{int(rate * 100)}.json", "w") as f:
    json.dump(raw_logs, f, indent=2)

print(json.dumps(summary_results, indent=2))
```

Final JSON output.

```
(hw5_venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\mcp_meals> python mcpGroceryRecall_toolContracts.py
[
  {
    "injected_failure_rate": "0%",
    "success_rate": "100.0%",
    "mean_latency_ms": 494.69,
    "p99_latency_ms": 915.29
  },
  {
    "injected_failure_rate": "20%",
    "success_rate": "100.0%",
    "mean_latency_ms": 567.24,
    "p99_latency_ms": 1177.49
  },
  {
    "injected_failure_rate": "50%",
    "success_rate": "96.0%",
    "mean_latency_ms": 563.56,
    "p99_latency_ms": 855.71
  }
]
```

Part 4

Implementing testing for all the successful and unsuccessful searchers from part 3 in the code

```
res1 = asyncio.run(execute_tool("search_grocery_recalls", {"query": "contamination"}))
data1 = json.loads(res1)
assert_test("Valid Search", data1["ok"] == True and len(data1["data"]) > 0, results)

res2 = asyncio.run(execute_tool("search_grocery_recalls", {"query": "The Avengers"}))
data2 = json.loads(res2)
assert_test("Invalid Search", data2["ok"] == False and data2["error"] is not None, results)

res3 = asyncio.run(execute_tool("recall_detail_lookup", {"recall_number": "F-0975-2021"}))
data3 = json.loads(res3)
assert_test("Valid Detail Lookup", data3["ok"] == True or data3["data"] is not None, results)

res4 = asyncio.run(execute_tool("recall_detail_lookup", {"recall_number": "67"}))
data4 = json.loads(res4)
assert_test("Invalid Detail Lookup", data4["ok"] == False, results)

res5 = asyncio.run(execute_tool("aggregate_company_stats", {"company_name": "Safeway"}))
data5 = json.loads(res5)
assert_test("Valid Aggregate Stats", data5["ok"] == True, results)

res6 = asyncio.run(execute_tool("aggregate_company_stats", {"company_name": "Pokemon"}))
data6 = json.loads(res6)
assert_test("Invalid Aggregate Stats", data6["ok"] == False, results)
```

Final test results output.

```
(hws_venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\mcp_meals> python mcpGroceryRecall_safeTool.py
[10/05/26 00:56:11] INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=product_description:contamination&limit=5 "HTTP/1.1 200 OK"
Test Passed: Valid Search
[10/05/26 00:56:12] INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=product_description:The%20Avengers&limit=5 "HTTP/1.1 404 Not Found"
Test Passed: Invalid Search
[10/05/26 00:56:12] INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=recall_number:F-0975-2021 "HTTP/1.1 200 OK"
Test Passed: Valid Detail Lookup
[10/05/26 00:56:13] INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=recall_number:67 "HTTP/1.1 404 Not Found"
Test Passed: Invalid Detail Lookup
[10/05/26 00:56:13] INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=recalling_firm:Safeway&limit=100 "HTTP/1.1 200 OK"
Test Passed: Valid Aggregate Stats
[10/05/26 00:56:13] INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=recalling_firm:Pokemon&limit=100 "HTTP/1.1 404 Not Found"
Test Passed: Invalid Aggregate Stats
Test Results: 6/6 tests passed
```

Part 5

Safety rules to prevent unauthorized or restricted records from being accessed.

```
query_value = str(inputs.get("query", "")).lower()
company_value = str(inputs.get("company_name", "")).lower()

if "banned" in query_value or "unauthorized" in company_value or "classified" in query_value:
    return json.dumps({"ok": False, "data": None, "error": "You are not authorized to access banned or restricted grocery recall records"})
```

Implementing all available tools to use for the agent loop

```
#all available tools
available_tools = [{"type": "function",
                    "function": {"name": "search_grocery_recalls", "description": "Search grocery recalls by product description query.",
                                "parameters": {"type": "object", "properties": {"query": {"type": "string"}}, "required": ["query"]}}},
                   {"type": "function",
                    "function": {"name": "recall_detail_lookup", "description": "Get detailed information about a specific recall by its recall number.",
                                "parameters": {"type": "object", "properties": {"recall_number": {"type": "string"}}, "required": ["recall_number"]}}},
                   {"type": "function",
                    "function": {"name": "aggregate_company_stats", "description": "Get aggregated recall statistics for a specific recalling company.",
                                "parameters": {"type": "object", "properties": {"company_name": {"type": "string"}}, "required": ["company_name"]}}}]
```

Agent loop

```
while step_count < max_steps:
    step_count += 1

    if client:
        response = client.chat(model="llama3", messages=messages, tools=available_tools)
    else:
        response = ollama.chat(model="llama3", messages=messages, tools=available_tools)

    message = response.get("message", {})
    messages.append(message)

    tool_calls = message.get("tool_calls", [])

    if not tool_calls:
        stop_reason = "model_finished"
        break

    for tool_call in tool_calls:
        tool_call_count += 1
        func_name = tool_call["function"]["name"]
        func_args = tool_call["function"]["arguments"]

        tool_result_json = asyncio.run(execute_tool(func_name, func_args))

        run_log_entries.append({"step": step_count, "tool_call": func_name, "input": func_args, "result": json.loads(tool_result_json)})
        messages.append({"role": "tool", "name": func_name, "content": tool_result_json})

    else:
        stop_reason = "max_steps_exceeded"
```

Additional tests implemented for safety rules and agent loop

```
allowed_demo = asyncio.run(execute_tool("search_grocery_recalls", {"query": "cheese"}))
blocked_demo = asyncio.run(execute_tool("search_grocery_recalls", {"query": "banned substance"}))

allowed_data = json.loads(allowed_demo)
blocked_data = json.loads(blocked_demo)

assert_test("Safety Rule Allowed Call", allowed_data["ok"] == True, results)
assert_test("Safety Rule Blocked Call", blocked_data["ok"] == False and "Safety violation" in blocked_data["error"], results)

safety_test_res = asyncio.run(execute_tool("aggregate_company_stats", {"company_name": "unauthorized firm"}))
safety_data = json.loads(safety_test_res)
assert_test("Offline Test - Safety Rule Enforcement", safety_data["ok"] == False, results)

mock_responses = [{"message": {"tool_calls": [{"function": {"name": "search_grocery_recalls", "arguments": {"query": "cheese"}}}]},
                  {"message": {"tool_calls": [{"function": {"name": "search_grocery_recalls", "arguments": {"query": "cheese"}}}]},
                  {"message": {"tool_calls": [{"function": {"name": "search_grocery_recalls", "arguments": {"query": "cheese"}}}]}} ]
mock_client = MockModel(mock_responses)
agent_output = run_agent("Find cheese recalls repeatedly", max_steps=2, client=mock_client)
assert_test("Offline Test - Max Steps Limit Reached", agent_output["stop_reason"] == "max_steps_exceeded", results)
```

Final test output.

```
(hws_venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\mcp_meals> python mcpGroceryRecall_safeTool.py
[10/05/26 11:09:56] INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=product_description:contamination&limit=5 "HTTP/1.1 200 OK" _client.py:1740
Test Passed: Valid Search
[10/05/26 11:09:57] INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=product_description:The%20Avengers&limit=5 "HTTP/1.1 404 Not Found" _client.py:1740
Test Passed: Invalid Search
INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=recall_number:F-0975-2021 "HTTP/1.1 200 OK" _client.py:1740
Test Passed: Valid Detail Lookup
[10/05/26 11:09:58] INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=recall_number:67 "HTTP/1.1 404 Not Found" _client.py:1740
Test Passed: Invalid Detail Lookup
INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=recalling_firm:Safeway&limit=100 "HTTP/1.1 200 OK" _client.py:1740
Test Passed: Valid Aggregate Stats
[10/05/26 11:09:59] INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=recalling_firm:Pokemon&limit=100 "HTTP/1.1 404 Not Found" _client.py:1740
Test Passed: Invalid Aggregate Stats
INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=product_description:cheese&limit=5 "HTTP/1.1 200 OK" _client.py:1740
Test Passed: Safety Rule Allowed Call
Test Failed: Safety Rule Blocked Call,
Test Passed: Offline Test - Safety Rule Enforcement
[10/05/26 11:10:00] INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=product_description:cheese&limit=5 "HTTP/1.1 200 OK" _client.py:1740
INFO HTTP Request: GET https://api.fda.gov/food/enforcement.json?search=product_description:cheese&limit=5 "HTTP/1.1 200 OK" _client.py:1740
Test Passed: Offline Test - Max Steps Limit Reached
Test Results: 9/10 tests passed
```

In the process of running this file, an offline test is executed. This will evaluate loop safeguards that are implemented for these tests. The MockModel class is utilized in this test to oversee everything. The tests will look through the MCP tools search_grocery_recalls, recall_detail_lookup, and aggregate_company_stats.

The harness starts out by sending the test inputs to MockModel class, which then targets search_grocery_recalls. Afterwards, the harness goes through execute_tool which starts a safety check of the input and ensures that nothing entered has any keywords that indicates it is a restricted or unauthorized record: all details are recorded.

After one test is done, the step count then increments, and the next test will be sent to MockModel. The harness will terminate execution if the max steps are exceeded.

Github

<https://github.com/Aywei2001/data260--SID4->